Experiment - 9

Student Name: Syed Adnan Hossain **UID:** 23BCS11343
Branch: BE-CSE **Section/Group:** KRG-3B
Semester: 6th
Subject Name: System Design **Subject Code:** 23CSH-314

Aim: To design a highly scalable **ticket booking platform** that allows users to discover and purchase tickets for entertainment events, ensuring high availability for event discovery and strict consistency for seat reservations.

Objectives:

- Understand the architecture of a massive-scale ticketing and reservation system.
- Identify functional requirements such as event discovery, real-time seat mapping, and booking execution.
- Identify non-functional requirements including handling 100 Million DAU, search availability, and transaction consistency.
- Analyze the decoupling of payment workflows and status updates using event-driven stream processing (Kafka).
- Design REST APIs for event metadata, seat layouts, and secure transaction

Procedure:

1. Studied real-world ticket booking architectures and concurrent transaction handling.
2. Identified core entities such as Users, Events, Venues, Shows, Seats, and Bookings.
3. Analyzed TTL-based distributed locking mechanisms to temporarily hold seats and prevent double-booking.
4. Collected functional and non-functional requirements for event discovery and high-demand ticket sales.
5. Designed APIs for search filtering, comprehensive event display, and booking execution.
6. Evaluated CQRS-like patterns by separating read-heavy search operations (via Elasticsearch and CDC pipelines) from write-heavy primary databases.
7. Analyzed asynchronous message brokers (Apache Kafka) for handling third-party payment callbacks, seat status updates, and user notifications.

Functional Requirements:

1. **Search & Discovery:** Users must search for events by title, city, or genre with fuzzy matching.
2. **Seat Selection:** Real-time visualization of seat maps with current availability status.
3. **Temporary Seat Locking:** Atomic locking of selected seats for 5 minutes to prevent race conditions.
4. **Booking & Payment:** Secure transaction processing with automated ticket generation upon success.
5. **Async Notifications:** Delivery of booking confirmation via Email/SMS post-transaction.

Non-functional Requirements:

1. **High Availability:** The browsing/searching flow must be 99.99% available.
2. **Strong Consistency:** Mandatory for the booking and seat-locking flow to prevent double bookings.
3. **Low Latency:** Search results should return in <100ms; seat locking should occur in <200ms.
4. **Scalability:** Ability to handle massive traffic spikes (100k+ TPS) during popular event releases.
5. **Fault Tolerance:** Use of circuit breakers for third-party payment gateways to prevent cascading failures.

Core Entities of the System:

- **User:** Profile, authentication credentials, and preferences.
- **Event/Show:** Movie/Concert metadata, venue mapping, and timing.
- **Seat:** Physical coordinate, category (VIP/Gold), and price.
- **Booking/Order:** Transactional record linking User, Show, and Seats.
- **SeatLock:** Temporary distributed lock entity (Redis-backed).

API Design:

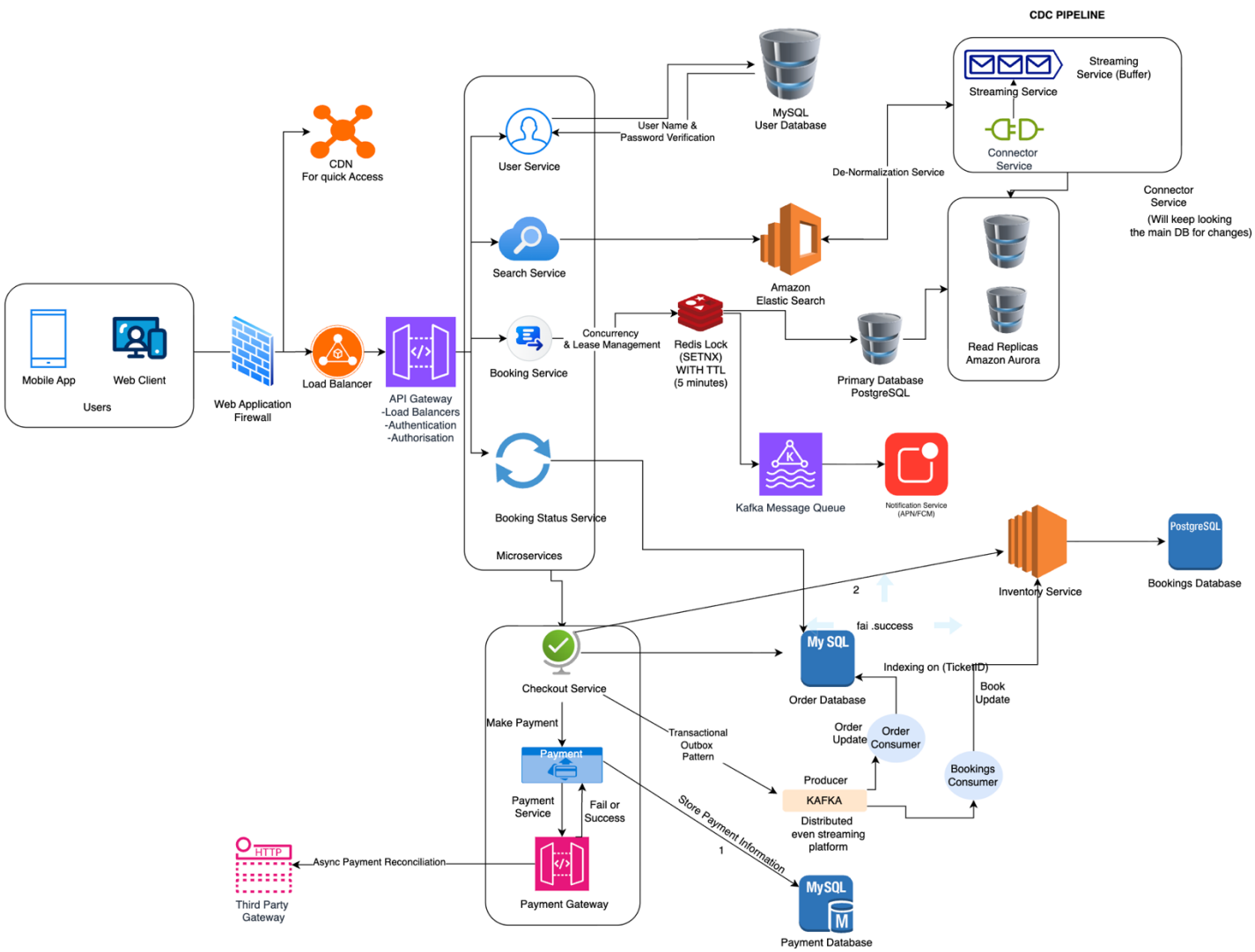
- `GET /v1/events?city={id}&q={query}`: Returns filtered events from ElasticSearch.
- `GET /v1/shows/{show_id}/seats`: Fetches real-time seat map from Read Replicas/-Cache.
- `POST /v1/seats/lock`: (Payload: `seat_ids[]`) Initiates Redis `SETNX` with a 300s TTL.
- `POST /v1/bookings`: Creates a pending order and initiates the payment redirect.
- `POST /v1/payments/webhook`: Asynchronous callback endpoint for payment gateway confirmation.

Database Schema Design:

User Table: (user_id, name, email, phone)
Show Table: (show_id, event_id, venue_id, start_time, base_price)
Seat Table: (seat_id, show_id, seat_num, status [Available, Locked, Booked])

Booking Table: (booking_id, user_id, show_id, total_amt, status [Pending, Paid, Failed])
Outbox Table: (event_id, payload, status) – *Used for Transactional Outbox Pattern to ensure Kafka sync.*

High Level Design (HLD):



The architecture follows a microservices pattern optimized for high-concurrency:

- **Global Edge:** CDN handles static assets; L7 Load Balancer manages traffic across API Gateways.
- **Concurrency Layer: Redis SETNX** acts as a distributed lock. If payment fails or the 5-min TTL expires, the lock is released automatically.
- **Search Strategy:** A **CDC (Change Data Capture)** pipeline via Kafka Connect streams SQL updates to **ElasticSearch** for sub-second searching.

- **Reliability:** The **Transactional Outbox Pattern** ensures that the Booking DB update and the Notification Service (Kafka) are always in sync.
- **Data Strategy:** Write-intensive booking operations hit the **Primary SQL DB**, while heavy seat-map reads are distributed across **Read Replicas**

Learning Outcomes:

- Designed a highly scalable ticket booking system with microservices architecture.
- Learned how real-time seat selection and reservation systems work.
- Understood concurrency control using distributed locking to prevent double booking.
- Implemented efficient API design for event discovery and booking workflows.
- Gained knowledge of event-driven systems using Kafka for payments and notifications.